

Progress Report

Day 9 — Task Module & Filtering System

Task Management MERN Stack Platform

Shafin Nigamana | 27 May 2026

1. Day-9 Objective

The goal for Day 9 was to implement the Task management module, create Task CRUD APIs, configure task relationships with Users and Teams, and add filtering support for task retrieval.

2. Tasks Completed

Task Schema Design

- Created Task.js model inside server/src/models
- Added fields:
 - title
 - description
 - assigneeId
 - teamId
 - status
 - priority
 - dueDate
- Configured:
 - status enum validation
 - priority enum validation
 - ObjectId references for User and Team relationships
 - timestamps

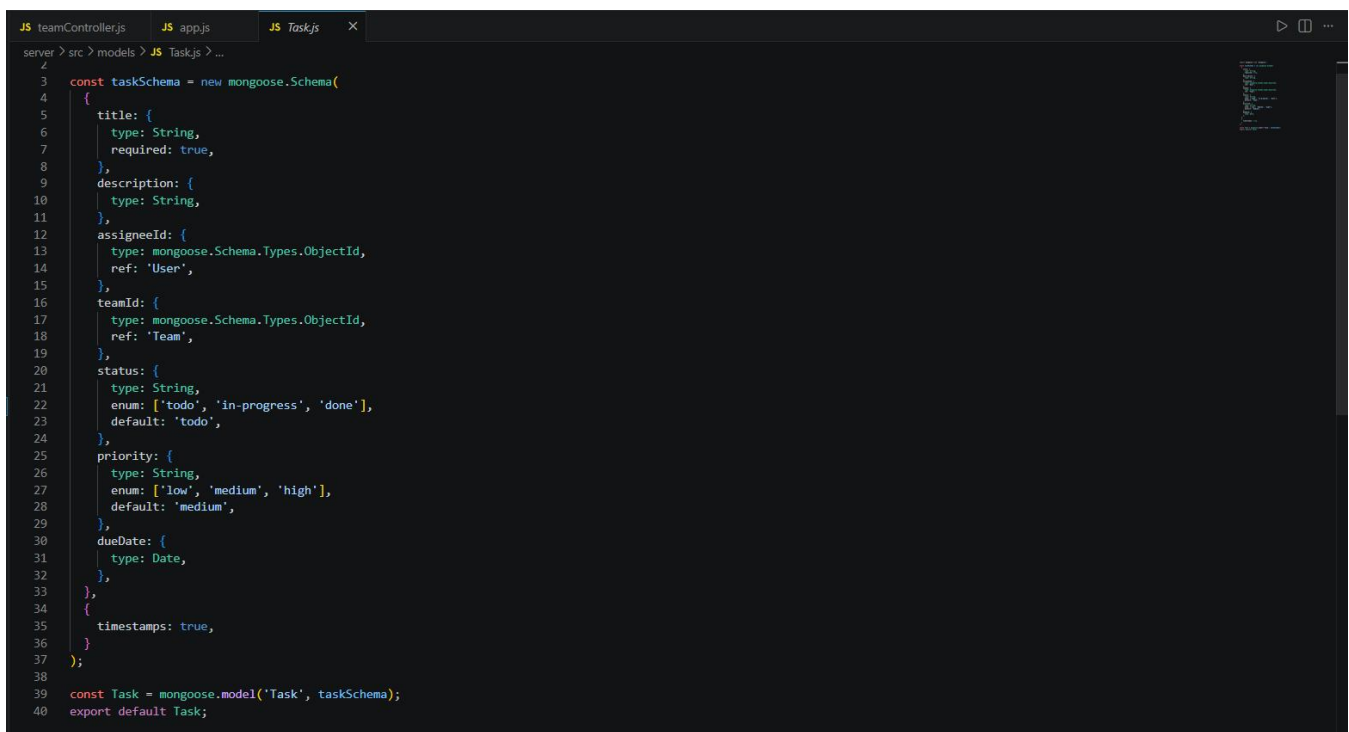


Figure 2.1 — Task schema created using Mongoose relationships.

Task CRUD Implementation

- Created taskController.js for task management operations
- Implemented:
 - create task
 - get tasks
 - get task by ID
 - update task
 - delete task
- Integrated Task model with MongoDB collection operations

```

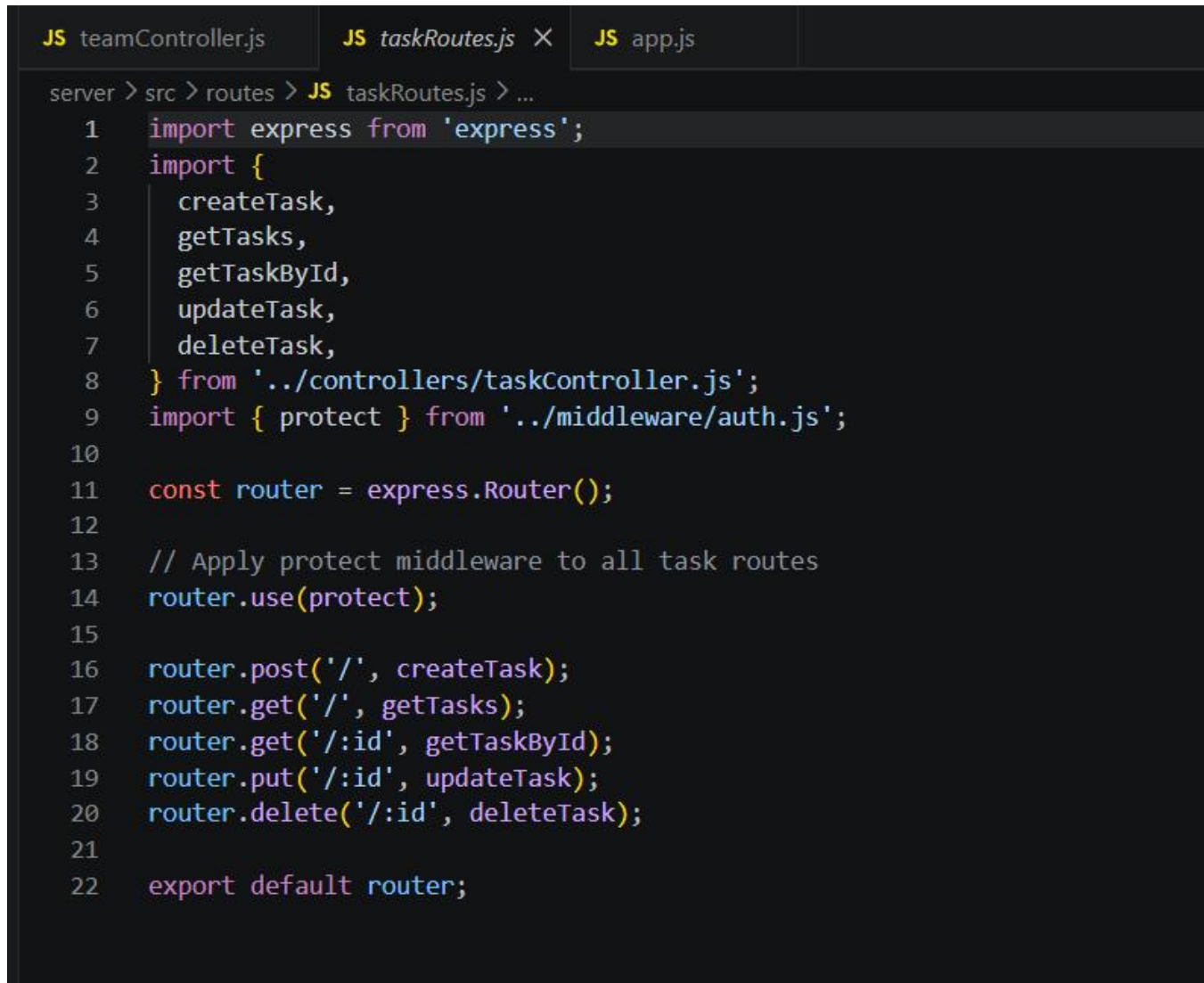
JS teamController.js JS taskController.js X JS app.js
server > src > controllers > JS taskController.js > ...
3
4 export const createTask = async (req, res) => {
5   try {
6     const { title, description, assigneeId, teamId, status, priority, dueDate } = req.body || {};
7
8     if (!title) {
9       return res.status(400).json({ message: 'Task title is required' });
10    }
11
12    const task = await Task.create({
13      title,
14      description,
15      assigneeId,
16      teamId,
17      status,
18      priority,
19      dueDate,
20    });
21
22    return res.status(201).json(task);
23  } catch (error) {
24    if (error.name === 'ValidationError') {
25      return res.status(400).json({ message: 'Invalid task data', error: error.message });
26    }
27    return res.status(500).json({ message: 'Error creating task', error: error.message });
28  }
29
30 export const getTasks = async (req, res) => {
31   try {
32     const { status, priority, assigneeId, teamId } = req.query;
33     const filter = {};
34
35     // Basic dynamic filtering based on query params
36     if (status) filter.status = status;
37     if (priority) filter.priority = priority;
38     if (assigneeId) filter.assigneeId = assigneeId;
39     if (teamId) filter.teamId = teamId;
40
41     const tasks = await Task.find(filter);
42     return res.status(200).json(tasks);
43   } catch (error) {
44     return res.status(500).json({ message: 'Error fetching tasks', error: error.message });
45   }
46
47 export const getTaskById = async (req, res) => {
48   try {
49     const { id } = req.params;
50     const task = await Task.findById(id);
51
52     if (!task) {
53       return res.status(404).json({ message: 'Task not found' });
54     }
55
56     return res.status(200).json(task);
57   } catch (error) {
58     return res.status(500).json({ message: 'Error fetching task', error: error.message });
59   }
60
61 export const updateTask = async (req, res) => {
62   try {
63     const { id } = req.params;
64     const updates = req.body || {};
65
66     const task = await Task.findByIdAndUpdate(
67       id,
68       updates,
69       { new: true, runValidators: true }
70     );
71
72     if (!task) {
73       return res.status(404).json({ message: 'Task not found' });
74     }
75
76     return res.status(200).json(task);
77   } catch (error) {
78     if (error.name === 'ValidationError') {
79       return res.status(400).json({ message: 'Invalid task data', error: error.message });
80     }
81     return res.status(500).json({ message: 'Error updating task', error: error.message });
82   }
83
84 export const deleteTask = async (req, res) => {
85   try {
86     const { id } = req.params;
87     const task = await Task.findByIdAndDelete(id);
88
89     if (!task) {
90       return res.status(404).json({ message: 'Task not found' });
91     }
92
93     return res.status(200).json({ message: 'Task deleted successfully' });
94   } catch (error) {
95     return res.status(500).json({ message: 'Error deleting task', error: error.message });
96   }
97
98
99
100

```

Figure 2.2 — Task CRUD controller implementation inside backend architecture.

Protected Task Routes

- Created taskRoutes.js for Task API endpoints
- Protected all Task routes using existing JWT authentication middleware
- Registered /api/tasks routes inside Express application



```
server > src > routes > JS taskRoutes.js > ...
1  import express from 'express';
2  import {
3    createTask,
4    getTasks,
5    getTaskById,
6    updateTask,
7    deleteTask,
8  } from '../controllers/taskController.js';
9  import { protect } from '../middleware/auth.js';
10
11  const router = express.Router();
12
13  // Apply protect middleware to all task routes
14  router.use(protect);
15
16  router.post('/', createTask);
17  router.get('/', getTasks);
18  router.get('/:id', getTaskById);
19  router.put('/:id', updateTask);
20  router.delete('/:id', deleteTask);
21
22  export default router;
```

Figure 2.3 — Protected Task routes integrated into backend routing structure.

Task Creation Verification

- Tested task creation endpoint using Postman
- Verified task creation with:
 - assigned user
 - related team

- status
- priority
- due date

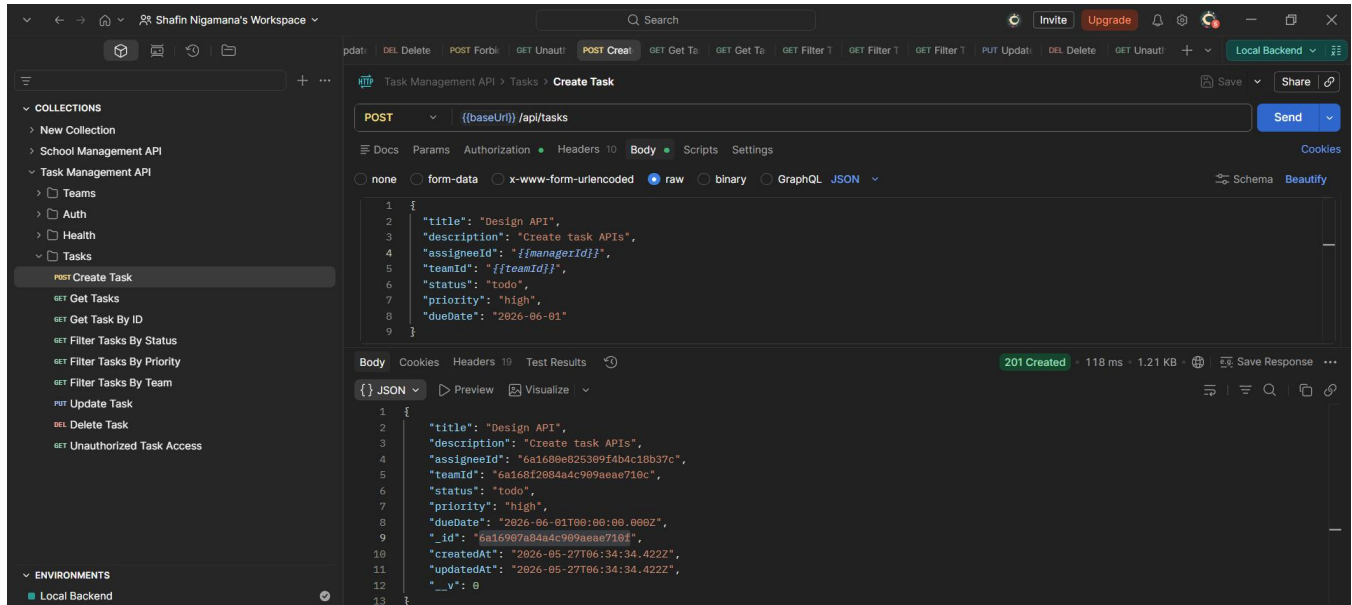


Figure 2.4 — Task creation endpoint tested successfully.

Task Filtering System

- Implemented filtering support inside GET /api/tasks endpoint
- Configured query-based filters for:
 - status
 - priority
 - assigneeId
 - teamId
- Verified filtered task retrieval using Postman

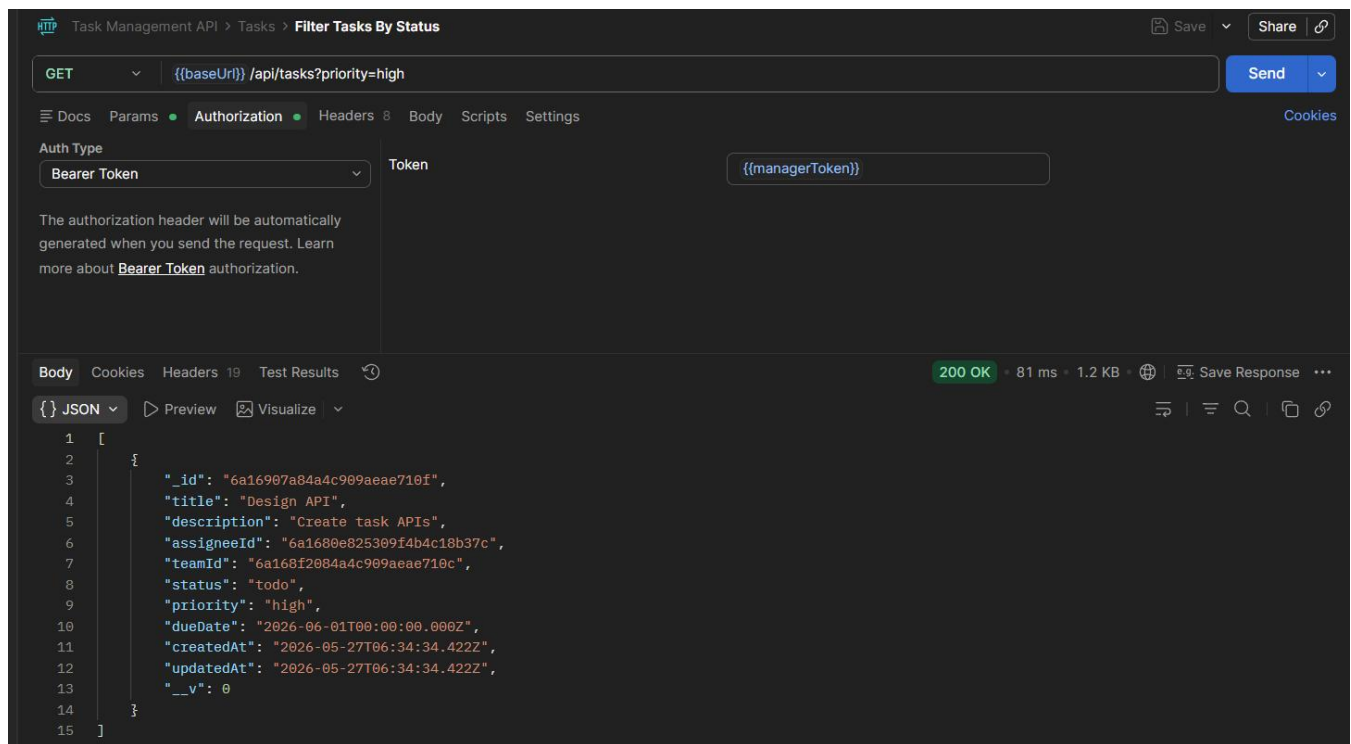


Figure 2.5 — Task filtering by status verified successfully.

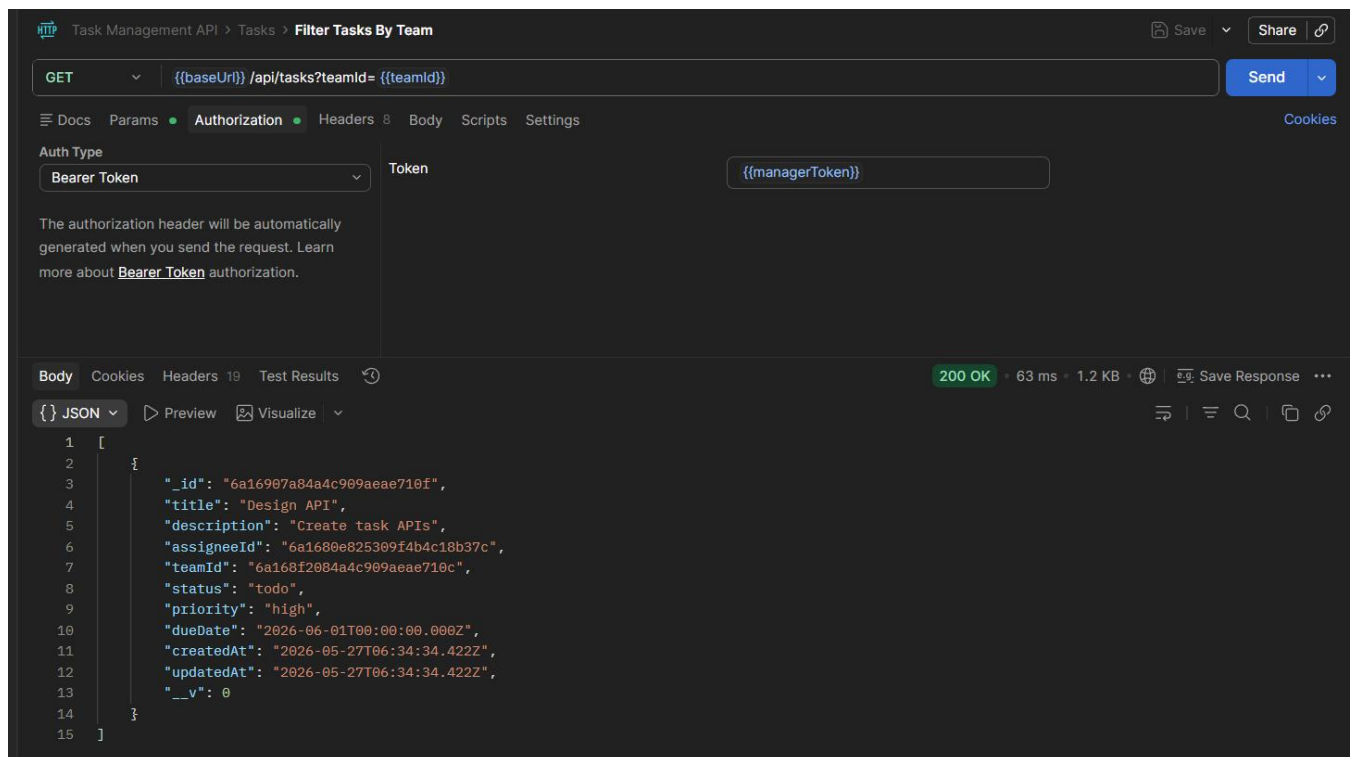


Figure 2.6 — Task filtering using team relationship verified successfully.

Task Update & Delete Verification

- Tested protected update and delete endpoints for tasks
- Verified backend updates and removes task data successfully

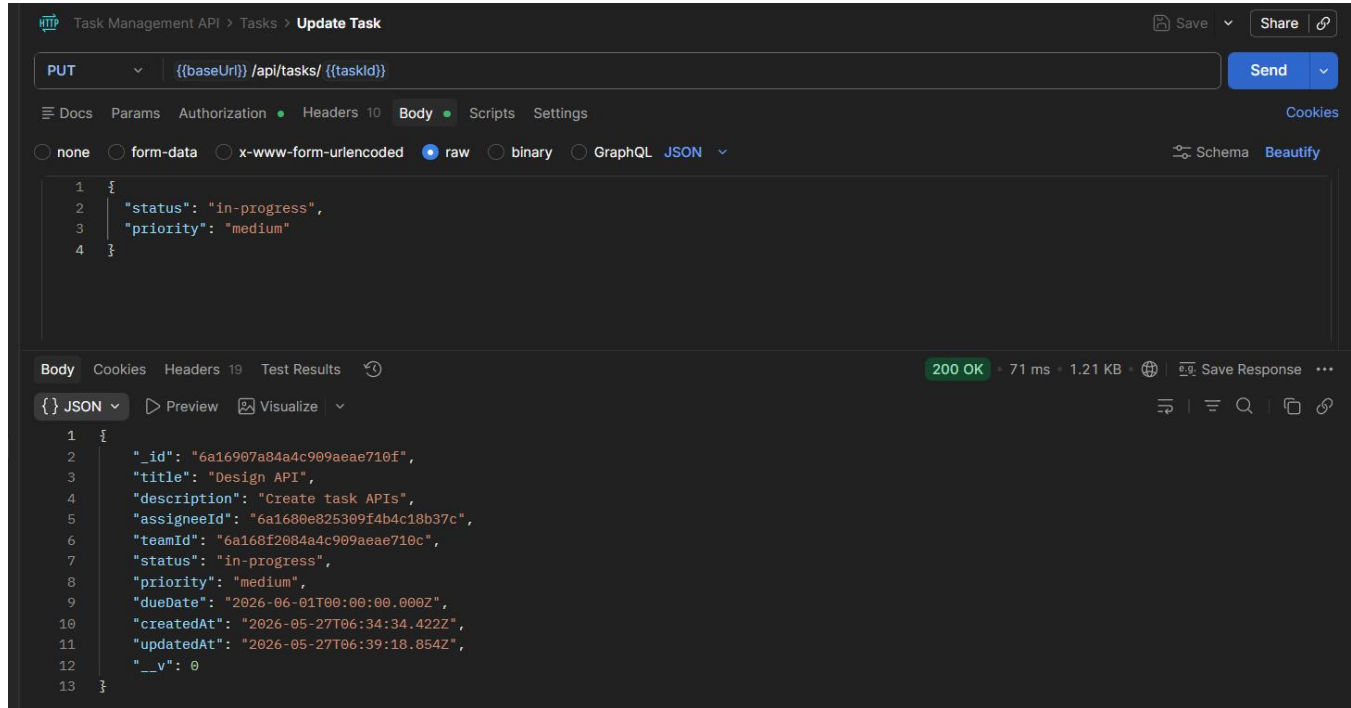


Figure 2.7 — Task update operation verified successfully.

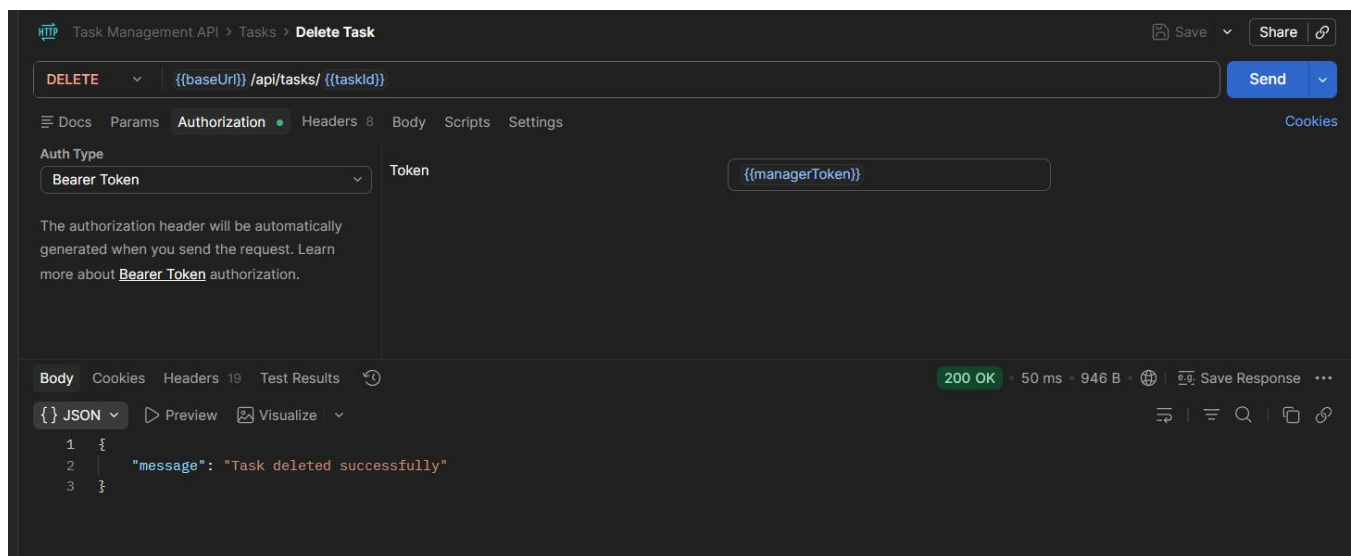


Figure 2.8 — Task delete operation verified successfully.

Unauthorized Access Verification

- Verified Task routes reject unauthenticated requests
- Confirmed JWT middleware protects Task APIs correctly

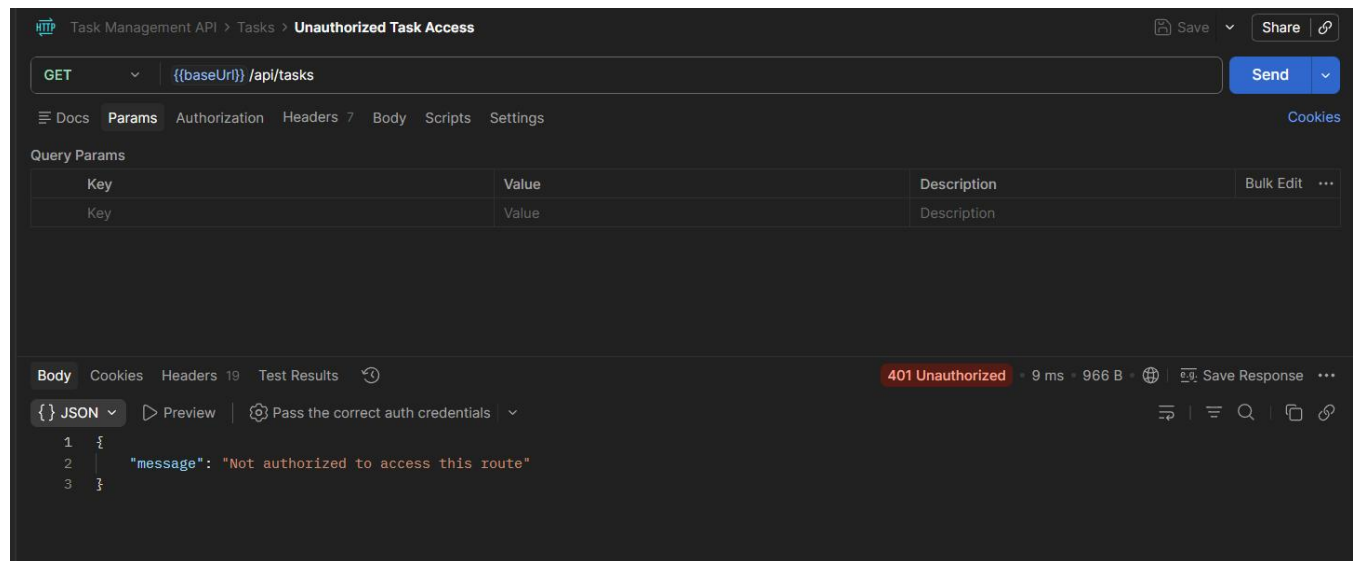


Figure 2.9 — Unauthorized request blocked from Task API access.

Postman Collection Update

- Added Task API requests into centralized Postman collection
- Configured reusable environment variables for:
 - taskId
 - teamId
 - managerId

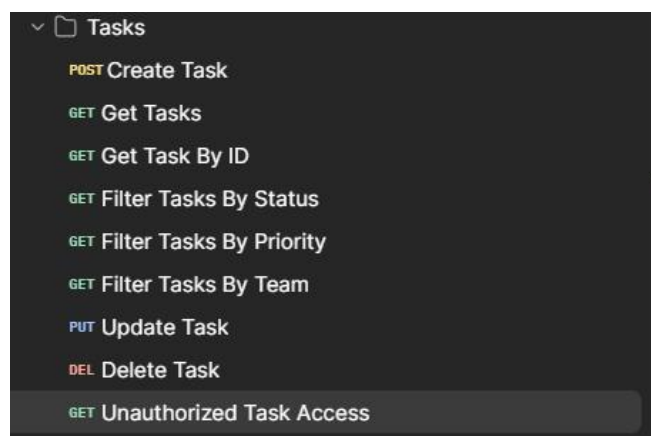


Figure 2.10 — Task API requests organized inside Postman collection.

Repository Update

- Committed Task module implementation changes using Git workflow
- Pushed latest backend Task module updates to GitHub repository

```
shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/task-module)
• $ git status
On branch feature/task-module
Your branch is up to date with 'origin/feature/task-module'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   docs/Postman/Task Management API.postman_collection.json
    modified:   server/src/app.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    server/src/controllers/taskController.js
    server/src/models/Task.js
    server/src/routes/taskRoutes.js

no changes added to commit (use "git add" and/or "git commit -a")

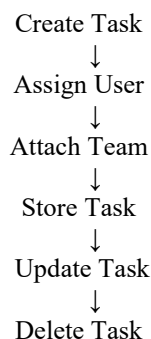
shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/task-module)
• $ git add .
warning: in the working copy of 'docs/Postman/Task Management API.postman_collection.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'server/src/app.js', LF will be replaced by CRLF the next time Git touches it

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/task-module)
• $ git commit -m "feat: implement task management module and filters"
[feature/task-module 0f9ec92] feat: implement task management module and filters
5 files changed, 429 insertions(+)
 create mode 100644 server/src/controllers/taskController.js
 create mode 100644 server/src/models/Task.js
 create mode 100644 server/src/routes/taskRoutes.js

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/task-module)
• $ git push
Enumerating objects: 24, done.
Counting objects: 100% (24/24), done.
Delta compression using up to 16 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (14/14), 2.79 KiB | 572.00 KiB/s, done.
Total 14 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
```

Figure 2.11 — Day 9 Task module implementation pushed to GitHub.

3. Current Task Flow



4. Next Steps (Day 10)

- Design MySQL audit_log table
- Implement audit logging helper for task operations
- Write audit entries for task create, update, and delete actions
- Prepare audit API endpoint structure

5. Conclusion

Day 9 focused on implementing the Task management module and task filtering system. Task CRUD APIs, protected Task routes, and query-based filters were implemented successfully using MongoDB relationships and JWT authentication middleware.

The backend now supports task management operations and is prepared for audit logging integration in the next phase.